BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Trần Ngọc Ánh
MSSV / Lớp	3122410013_ DCT1221

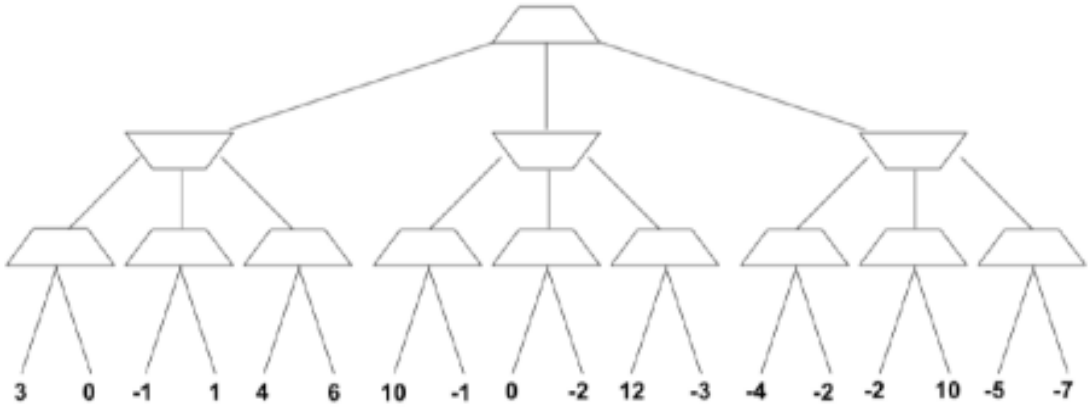
Mục tiêu học tập

• Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa. • Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
• Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP. • Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi. • Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

- Gọi: Nút gốc: A
  Nút tầng 1: B, C, D
  Nút tầng 2: E, F, G con của B; H, I, J con của C; K, L, M con của D
- Giá trị tại các nút tầng 2:
  E = max(3, 0) = 3
  F = max(-1, 1) = 1
  G = max(4, 6) = 6
  H = max(10, -1) = 10
  I = max(0, -2) = 0
  J = max(12, -3) = 12
  K = max(-4, -2) = -2
  L = max(-2, 10) = 10

M = max (-5, -7) = -5

Giá trị tại các nút tầng 1:

B = min(E, F, G) = min(3, 1, 6) = 1

C = min(H, I, J) = min(10, 0, 12) = 0

D = min(K, L, M) = min(-2, 10, -5) = -5

Giá trị tại nút gốc:

A = max(B, C, D) = max(1, 0, -5) = 1

b) MAX nên chọn nhánh ngoài cùng bên trái (nút B) vì nhánh có giá trị lớn nhất là 1, so với nhánh giữa là 0 và nhánh phải là -5. Giá trị của trò chơi là 1.

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Duyệt với α (MAX) và β (MIN) khởi tạo ở nút gốc là $\alpha = -\infty, \beta = +\infty$:

- Xét nhánh trái B:

Nút E (MAX) đánh giá lá 3 và 0 → E = 3. Nút B (MIN) cập nhật $\beta = 3$.

Nút F (MAX) đánh giá lá -1 và 1 → F = 1. Nút B cập nhật $\beta = \min(3,1) = 1$.

Nút G (MAX) đánh giá lá đầu tiên là 4. Vì $4 \geq \beta$ của B (4 $\geq$ 1), G sẽ trả về ít nhất là 4, nhưng B chỉ quan tâm các giá trị ≤ 1. Nhánh này bị cắt tỉa → Đánh dấu X nhánh đi xuống lá 6. B chốt giá trị 1.

Nút gốc A (MAX) cập nhật $\alpha = \max(-\infty, 1) = 1$.

- Xét nhánh giữa (C): A truyền $\alpha = 1, \beta = +\infty$ xuống.

Nút H (MAX) đánh giá lá 10 và -1 → H = 10. Nút C (MIN) cập nhật $\beta = 10$.

Nút I (MAX) đánh giá lá 0 và -2 → I = 0. Nút C cập nhật lại giá trị thành 0 (vì min (10,0) = 0 .

Tại C, hiện có giá trị 0. Vì $0 \leq \alpha$ của A (0 $\leq$ 1), C chắc chắn trả về giá trị ≤ 0 và sẽ không bao giờ được MAX chọn. Nhánh này bị cắt tỉa → Đánh dấu X toàn bộ nhánh đi xuống nút J (gồm lá 12 và -3). C chốt giá trị 0.

- Xét nhánh phải (D): A truyền $\alpha = 1, \beta = +\infty$ xuống.

Nút K (MAX) đánh giá lá -4 và -2 → K = -2.

Tại D (MIN), giá trị tạm thời là -2. Vì $-2 \leq \alpha$ của A (-2 $\leq$ 1), D chắc chắn không được MAX chọn. Nhánh này bị cắt tỉa → Đánh dấu X toàn bộ nhánh đi xuống nút L (lá -2, 10) và nút M (lá -5, -7).

d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

- Tổng cộng có 7 nút lá bị cắt tỉa không cần thăm, bao gồm:

Lá 6

Lá 12 và -3

Lá -2 và 10 (dưới nút L)

Lá -5 và -7 (dưới nút M)

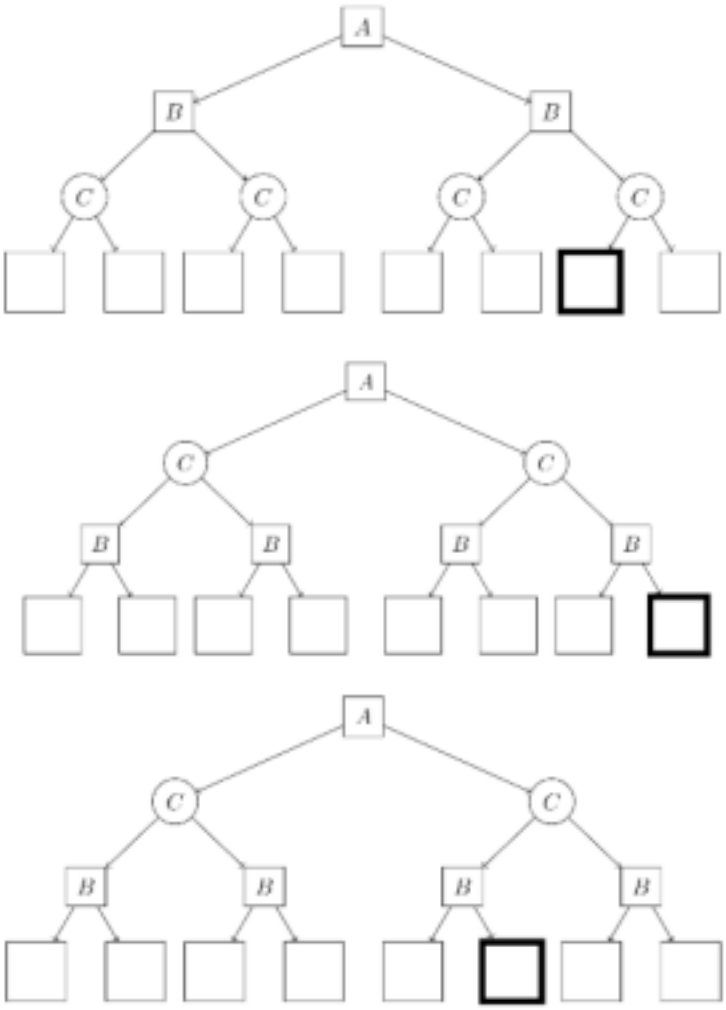
- So sánh:

Minimax bắt buộc phải duyệt và đánh giá toàn bộ 18 nút lá.

Alpha-Beta Pruning chỉ cần đánh giá 11 nút lá. Thuật toán này giúp giảm thiểu đáng kể số lượng nút phải tính toán (tiết kiệm 7 nút) mà vẫn đảm bảo tìm ra kết quả tối ưu tương tự như Minimax.

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

- Giá trị gán cho các lá từ trái qua phả): 0, 0, 0, 0, 10, 10. Nút viền đậm để trống.
- Điều kiện cắt tỉa: Nút gốc A là nút MIN và tầng B là nút MAX. Khi duyệt xong nhánh B bên trái, A nhận được giá trị ngưỡng $\beta = \max(0.5(0+0), 0.5(0+0)) = 0)$. Khi duyệt sang nhánh B bên phải, nút C thứ nhất (nút ngẫu nhiên) trả về giá trị kỳ vọng là 0.5(10 + 10) = 10. Vì B là nút MAX, giá trị của nhánh B bên phải chắc chắn ≥ 10. Do $10 > \beta = 0$, nút A (MIN) chắc chắn sẽ không chọn nhánh B phải. Toàn bộ nút C thứ hai (chứa nút viền đậm) bị cắt tỉa.

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

- Giá trị gán cho các lá từ trái qua phải: 0, 0, 0, 0, 20, 20, 0. Nút viền đậm để trống.
- Điều kiện cắt tỉa: Nút A là MIN, nút C là ngẫu nhiên và nút B là MAX. Duyệt xong nhánh C bên trái, A có ngưỡng $\beta = 0.5(\max(0,0)) + 0.5(\max(0,0)) = 0$. Sang nhánh C bên phải, B đầu tiên (MAX) trả về max (20,20) = 20. Ở B thứ hai (MAX), lá đầu tiên có giá trị 0, nên giá trị của B này chắc chắn ≥ 0. Nút C bên phải tính giá trị

kỳ vọng với các con có xác suất bằng nhau, do đó $C_{phải} \geq 0.5(20) + 0.5(0) = 10$. Vì $10 > \beta = 0$, phần còn lại của B thứ hai không thể làm C nhỏ hơn 0, nên nút viền đậm bị cắt tỉa

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

- Không thể cắt tỉa.
- Giải thích: Nút viền đậm nằm ở nhánh ngoài cùng bên trái của cây. Thuật toán duyệt từ trái sang phải, nên tại thời điểm xét nút này, gốc A (MIN) chưa duyệt bất kỳ nhánh nào khác để hình thành các giới hạn α hay β. Nút C (ngẫu nhiên) bắt buộc phải tính chính xác giá trị trung bình để truyền lên trên, nên nút B (MAX) bên dưới nó buộc phải đánh giá mọi nút lá.

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

- Trong Alpha-Beta Minimax thông thường, giá trị của một nút chỉ phụ thuộc vào cực trị (MAX hoặc MIN), nên khi phát hiện một nhánh con phá vỡ giới hạn α hoặc β, ta có thể cắt tỉa ngay các con còn lại bất kể giá trị của chúng là bao nhiêu.
- Trong Expinimax, nút ngẫu nhiên tính giá trị trung bình cộng của tất cả các nhánh con. Một nhánh chưa xét có thể mang giá trị dương vô cùng hoặc âm vô cùng, làm thay đổi hoàn toàn kết quả kỳ vọng. Do đó, việc cắt tỉa xuyên qua nút ngẫu nhiên chỉ khả thi khi ta có thể tính được **cận dưới hoặc cận trên** của tổng kỳ vọng bán phần (thông qua các nút MAX/MIN nằm dưới nó) và chứng minh được mức cận này đã đủ để nhánh bị từ chối bởi các giới hạn từ trên truyền xuống.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) $V_M \leq V_E$.

☑ Đúng ☐ Sai

Giải thích:

- V_M là giá trị khi các nút MIN chọn hành động có lợi nhất cho đối thủ, tức là lấy giá trị nhỏ nhất.
- V_E là giá trị kỳ vọng khi các nút MIN được thay bằng nút ngẫu nhiên, tức là tính trung bình có trọng số theo xác suất.

Với các giá trị con $x_1, x_2, ..., x_n$, ta luôn có:

$\min(x_1, ..., x_n) \leq \sum_{i=1}^{n} p_i x_i$ với $p_i \geq 0$ và $\sum p_i = 1$.

Do đó, giá trị kỳ vọng không nhỏ hơn giá trị nhỏ nhất, suy ra $V_M \leq V_E$.

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M.

☑ Đúng ☐ Sai

Giải thích:

Minimax chọn hành động tối ưu dựa trên giả định đối thủ luôn chọn nước đi bất lợi nhất cho người chơi MAX. Khi thay các nút MIN bằng nút ngẫu nhiên, đối thủ không còn chủ động chọn nước đi tệ nhất mà kết quả phụ thuộc vào

xác suất. Theo câu a, giá trị kỳ vọng trong cây trò chơi không thấp hơn giá trị Minimax tương ứng. Vì vậy, hành động được chọn theo Minimax vẫn đảm bảo lợi ích kỳ vọng ít nhất là V_M.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E.

☑ Sai ☐ Đúng

Minimax tối ưu hóa giá trị trong trường hợp xấu nhất, không tối ưu hóa giá trị kỳ vọng.

Trong khi đó, Expectimax lựa chọn hành động dựa trên giá trị kỳ vọng của các kết quả ngẫu nhiên. Vì hai chính sách tối ưu theo hai tiêu chí khác nhau, hành động Minimax chọn không nhất thiết đạt được giá trị kỳ vọng tối ưu V_E. Do đó, không thể đảm bảo lợi ích kỳ vọng ít nhất là V_E.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

- ☐ Minimax với tiện ích trạng thái cuối +1/−1
- ☑ Expectimax với tiện ích trạng thái cuối +1/−1
- ☑ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua
- ☐ Không phương án nào

d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

- Expectimax: Phù hợp khi kết quả có yếu tố ngẫu nhiên. Thuật toán tính giá trị kỳ vọng của các kết quả dựa trên xác suất xảy ra.
- Value Iteration: Phù hợp nếu mô hình trò chơi được biểu diễn thành một MDP, có trạng thái, hành động, xác suất chuyển trạng thái và phần thưởng. Có thể đặt phần thưởng 0 ở trạng thái trung gian, $+1$ khi thắng và -1 khi thua.

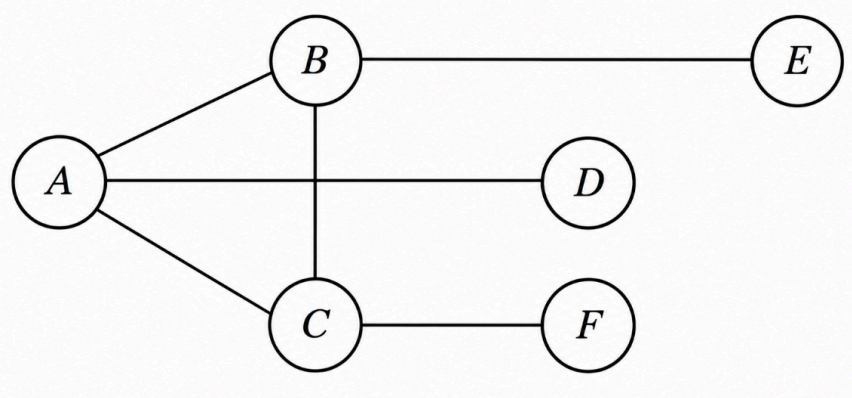
Vì sao không chọn Minimax? Minimax giả định có đối thủ chủ động chọn hành động chống lại người chơi. Đề bài mô tả trò chơi một người và kết quả không xác định trước, nên Expectimax hoặc Value Iteration phù hợp hơn.

Điều kiện cần lưu ý: Value Iteration cần mô hình chuyển trạng thái và phần thưởng đủ rõ ràng; Expectimax cần biết xác suất của các kết quả ngẫu nhiên.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).



b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Forward Checking chỉ kiểm tra các biến chưa được gán và có ràng buộc trực tiếp với biến vừa gán. Với A có các

biến liên quan là B, C, D.
Đáp án: B, C, D.

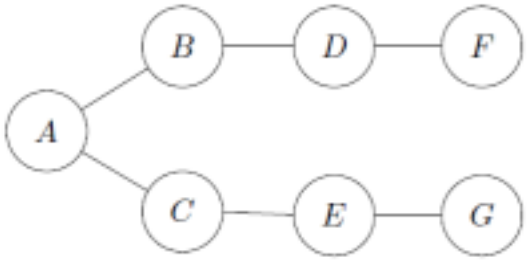
c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?
Theo danh sách ràng buộc, B liên quan trực tiếp với A, C, E.

- A đã được gán nên không xét miền của A.
- C và E chưa được gán nên miền của chúng có thể bị thu hẹp.

Đáp án: C, E.

d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?
Arc Consistency có thể lan truyền ảnh hưởng qua nhiều cạnh, không chỉ các biến kề trực tiếp với A. Từ A, ảnh hưởng có thể lan truyền qua các cạnh đến B, C, D, rồi đến E và F tùy theo các ràng buộc.
Đáp án: B, C, D, E, F.

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?
Khi gán B, ảnh hưởng có thể lan truyền qua các cạnh nối với B, sau đó tiếp tục qua đồ thị.
Đáp án: C, D, E, F.
Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

f) Thứ tự A → B → C → D → E → F → G
Không phải BFS cũng không phải DFS chuẩn.

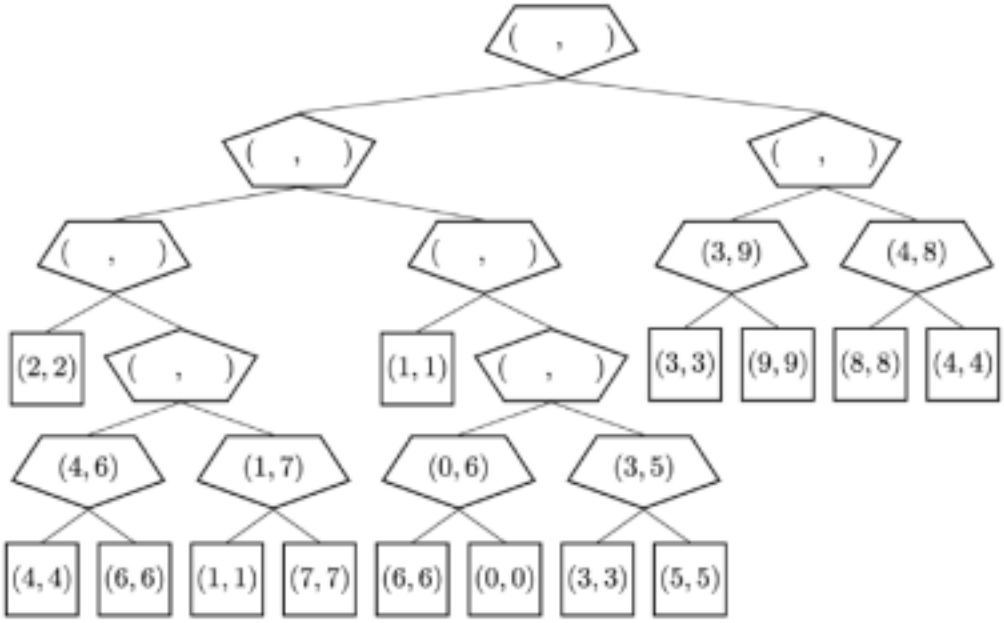
- BFS cần duyệt các đỉnh cùng mức trước: A, B, C, D, E, F, G.
- DFS đi sâu hết một nhánh trước khi quay lại: A, B, D, F, C, E, G.

g) Thứ tự G → E → C → A → B → D → F
DFS.
Bắt đầu tại G, đi sâu theo nhánh G → E → C → A, sau đó quay lại và đi tiếp nhánh A → B → D → F

h) Thứ tự G → F → E → D → C → B → A
Không phải BFS cũng không phải DFS chuẩn.
Vì G và F nằm ở hai nhánh khác nhau, không có cạnh nối trực tiếp giữa chúng. Nếu duyệt từ G thì phải đi qua E, C, A mới có thể sang nhánh chứa F.

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

Pacman chọn nút con có giá trị u lớn nhất và truyền cặp giá trị của nút được chọn lên nút cha.

Nút có hai con $(2, 2)$ và $(4, 6)$: chọn $(4, 6)$ vì $4 > 2$.

Nút có hai con $(1, 1)$ và $(1, 7)$: hai giá trị u bằng nhau. Có thể chọn một trong hai; nếu ưu tiên nút bên trái thì kết quả là $(1, 1)$.

Nút có hai con $(3, 9)$ và $(4, 8)$: chọn $(4, 8)$ vì $4 > 3$.

Hai nhánh của nút gốc có kết quả là (4,6) và (4,8). Casper thực tế muốn tăng v, nên chọn nhánh có $v = 8$. Đồng thời, Pacman vẫn tin rằng Casper giảm u, do đó giá trị u tại gốc là 4. Vậy **nút gốc có cặp giá trị (4, 8)**

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Tại nút Pacman có hai con (1,1) và nút Casper có hai lá (1,1), (7,7):

- Duyệt nút con trái (1,1), khi đó Pacman đã có giá trị (u=1).
- Xét nút Casper bên phải. Lá đầu tiên có giá trị (1,1), nên Casper có thể đạt (u=1).
- Khi đó (alpha = beta = 1), thỏa mãn điều kiện cắt tỉa.
- Không cần duyệt lá (7,7) vì kết quả của nhánh này không thể cải thiện quyết định của Pacman tại nút cha.

Do đó, theo thứ tự duyệt trái sang phải, có thể đánh dấu nhánh dẫn đến lá (7,7) là nhánh được cắt tỉa.

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Nếu Pacman biết Casper thực sự thân thiện và muốn tăng tiện ích cho Pacman, Pacman sẽ không còn đánh giá Casper theo giả định rằng Casper cố giảm tiện ích. Khi đó, cả Pacman và Casper đều hướng đến việc tối đa hóa tiện ích thực tế (v).

- Tại nút Pacman: chọn nút con có (v) lớn nhất.
- Tại nút Casper: cũng chọn nút con có (v) lớn nhất.

Tính từ dưới lên:

- (4,4) và (6,6) cho giá trị (6).

- (1,1) và (7,7) cho giá trị (7).
- (6,6) và (0,0) cho giá trị (6).
- (3,3) và (5,5) cho giá trị (5).
- (3,3) và (9,9) cho giá trị (9).
- (8,8) và (4,4) cho giá trị (8).

Các nút Pacman ở tầng trên lần lượt chọn giá trị lớn nhất:

- (max(2,4) = 4).
- (max(1,7)=7).
- (max(3,9)=9).
- (max(8,4)=8).

Tiếp tục lên tầng trên, hai nhánh cho giá trị lần lượt là (7) và (9). Cuối cùng, nút gốc chọn giá trị lớn nhất: [max(7,9)=9]

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	Đầy đủ nếu hệ số phân nhánh b hữu hạn.	Tối ưu nếu chi phí các bước bằng nhau (mỗi bước có cùng chi phí).
Depth-First Search (DFS)	Không đầy đủ trong không gian trạng thái vô hạn hoặc có chu trình nếu không kiểm tra trạng thái lặp. Đầy đủ nếu không gian hữu hạn và có cơ chế tránh lặp phù hợp.	Không tối ưu.
Iterative Deepening Search (IDS)	Đầy đủ nếu hệ số phân nhánh b hữu hạn và tồn tại lời giải ở độ sâu hữu hạn.	Tối ưu nếu chi phí các bước bằng nhau.
A* Search	Đầy đủ nếu hệ số phân nhánh hữu hạn, chi phí mỗi bước không nhỏ hơn một số dương $\varepsilon > 0$, và thuật toán xử lý trạng thái lặp phù hợp.	Tối ưu nếu heuristic $h(n)$ chấp nhận được (*admissible*). Với đồ thị, thường cần heuristic nhất quán (*consistent*) hoặc cơ chế mở lại nút đã đóng khi tìm được đường đi tốt hơn.

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.

* Chi phí bước

Chi phí bước quyết định mối quan hệ giữa độ sâu của lời giải và tổng chi phí. Ví dụ, có hai đường đi:

- Đường A: 2 bước, mỗi bước có chi phí 10 → tổng chi phí \(20\).
- Đường B: 3 bước, mỗi bước có chi phí 1 → tổng chi phí \(3\).

BFS chọn đường A vì đường này có ít bước hơn, nhưng đường B mới có tổng chi phí thấp nhất.

Vì vậy, BFS và IDS chỉ đảm bảo tối ưu khi mọi bước có chi phí bằng nhau. Khi chi phí khác nhau, số bước ít nhất không đồng nghĩa với chi phí thấp nhất.

* Đặc tính heuristic

Heuristic ảnh hưởng đến cách A* ưu tiên mở rộng các nút.

- Nếu heuristic chấp nhận được, A* có thể bảo đảm tìm lời giải tối ưu khi thuật toán đáp ứng các điều kiện cần thiết.
- Nếu heuristic đánh giá quá cao chi phí còn lại, A* có thể ưu tiên nhánh không phù hợp và tìm thấy lời giải không tối ưu.
- Nếu h(n)=0 với mọi nút, A* trở thành thuật toán tìm kiếm chi phí đồng nhất (Uniform-Cost Search), ưu tiên nút có (g(n)) nhỏ nhất.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Khi chi phí cạnh tăng từ *c* lên c + n, tổng chi phí của đường đi tối ưu cũ tăng thêm n. Do đó, đường đi cũ có thể không còn tối ưu vì có thể tồn tại một đường đi khác có chi phí thấp hơn.

Cách xử lý: Cập nhật chi phí cạnh, tính lại chi phí các nút bị ảnh hưởng trên đường đi cũ, sau đó tiếp tục tìm kiếm để kiểm tra các đường đi thay thế. Không nhất thiết phải chạy lại A* từ đầu nếu có thể tận dụng thông tin đã khám phá.

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Khi chi phí cạnh giảm từ *c* xuống c − n, tổng chi phí đường đi tối ưu cũ giảm n. Vì chi phí các cạnh khác không đổi và cạnh này thuộc đường đi tối ưu cũ, đường đi đó vẫn tối ưu; chi phí tối ưu mới giảm n.

Cách xử lý: Cập nhật chi phí cạnh và giá trị g của các nút nằm phía sau cạnh X → Y trên đường đi. Cập nhật các giá trị f = g + h tương ứng. Không cần tìm lại từ đầu nếu không có thay đổi nào khác.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Vì cạnh X > Y không thuộc đường đi tối ưu cũ nên việc tăng chi phí cạnh không làm thay đổi chi phí của đường đi tối ưu hiện tại. Hơn nữa, việc tăng chi phí một cạnh không thuộc đường tối ưu không thể làm cho đường đi qua cạnh đó trở nên tốt hơn so với trước.

Cách xử lý: Cập nhật chi phí cạnh trong đo thị. Đường đi ti ưu cũ vẫn là đương đi tối ưu vì chi phí của các đường đi khác chỉ giữ nguyên hoặc tăng lên. Không cần chạy lại A* từ đầu.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Khi chi phí cạnh giảm, đường đi chứa cạnh X → Y có thể trở nên tốt hơn và có khả năng thay thế đường đi tối ưu cũ. Tuy nhiên, không thể khẳng định chắc chắn đường đi tối ưu sẽ thay đổi, vì chi phí mới có thể vẫn lớn hơn hoặc bằng chi phí của đường đi tối ưu hiện tại.

Cách xử lý: Cập nhật chi phí cạnh, tính lại chi phí các đường đi liên quan đến cạnh này và so sánh với chi phí tối ưu hiện tại. Nếu tìm được đường đi có chi phí thấp hơn thì cập nhật đường đi tối ưu mới; nếu không, giữ nguyên đường đi cũ.

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Vì cạnh này chưa được khám phá và không thuộc đường đi tối ưu cũ, việc tăng chi phí không ảnh hưởng đến chi phí của đường đi tối ưu hiện tại. Việc tăng chi phí cũng không thể tạo ra một đường đi mới tốt hơn thông qua cạnh này.

Cách xử lý: Cập nhật chi phí cạnh trong đồ thị. Có thể giữ nguyên đường đi tối ưu hiện tại mà không cần chạy lại A* từ đầu.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Việc giảm chi phí cạnh có thể tạo ra một đường đi mới có chi phí thấp hơn đường đi tối ưu hiện tại. Do cạnh chưa được khám phá, thuật toán chưa có đủ thông tin về khả năng này.

Cách xử lý: Cập nhật chi phí cạnh, khám phá cạnh và đánh giá các đường đi có thể đi qua X → Y. Nếu phát hiện đường đi có chi phí thấp hơn thì cập nhật đường đi tối ưu mới; nếu không, giữ nguyên đường đi hiện tại.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

- A: act(percept) → f_A()
- B: act(percept) → f_B(percept)
- C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Chọn B và C.

Giải thích:

Môi trường quan sát đầy đủ nghĩa là percept hiện tại cung cấp đầy đủ thông tin cần thiết về trạng thái môi trường. Môi trường xác định nghĩa là kết quả của hành động được xác định bởi trạng thái hiện tại và hành động đó. Vì vậy, tác tử B có thể chọn hành động hợp lý chỉ dựa vào percept hiện tại.

Tác tử C cũng có thể hành động hợp lý vì lưu lịch sử percept không gây trở ngại; nó có thể sử dụng thông tin trạng thái hiện tại để lựa chọn hành động.

Tác tử A không sử dụng percept nên không thể bảo đảm chọn hành động hợp lý trong mọi môi trường thuộc lớp này.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Chọn C.

Giải thích:

Trong môi trường quan sát một phần, percept hiện tại không cung cấp đầy đủ thông tin về trạng thái thực tế của môi trường. Vì vậy, chỉ dựa vào percept hiện tại có thể không đủ để xác định hành động tốt nhất.

Tác tử C lưu lại toàn bộ lịch sử percept nên có thể kết hợp các quan sát trước đó để suy luận về trạng thái môi trường và lựa chọn hành động phù hợp.

Tác tử B chỉ dựa vào percept hiện tại nên không thể bảo đảm hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp này. Tác tử A không sử dụng percept nên cũng không thể bảo đảm điều đó.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Chọn C.

Giải thích:

Môi trường vừa quan sát một phần vừa ngẫu nhiên khiến tác tử không thể xác định chắc chắn trạng thái hiện tại chỉ từ một percept, đồng thời kết quả hành động có thể không xác định trước.

Tác tử C có thể sử dụng lịch sử percept để cập nhật hiểu biết về trạng thái môi trường, kết hợp với mô hình xác suất và hàm đánh giá hiệu suất để lựa chọn hành động có lợi ích kỳ vọng cao nhất.

Tác tử B chỉ sử dụng percept hiện tại nên có thể thiếu thông tin cần thiết để đưa ra quyết định tối ưu. Tác tử A

không sử dụng percept nên không thể bảo đảm hành động hợp lý trong mọi môi trường thuộc lớp này.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.
Đúng.
Giải thích:
Có thể tồn tại một môi trường mà mọi hành động đều mang lại hiệu suất như nhau. Khi đó, bất kể tác tử A, B hay C lựa chọn hành động nào, kết quả đều đạt mức hiệu suất tối đa có thể đạt được. Vì vậy, tất cả các tác tử đều hành động hợp lý trong môi trường đó.
Ví dụ: Một môi trường chỉ có một hành động hợp lệ và hành động đó luôn dẫn đến cùng một kết quả. Khi ấy, cả ba tác tử đều thực hiện hành động hợp lý, bất kể cách chúng xử lý percept khác nhau như thế nào.

— Hết —